

DSA STUDY BLUEPRINT SERIES

# 86. Transform to Sum Tree

Converting Tree to Cumulative Child Sum values

Section 09 • C++ Core Data Structures & Algorithms

# Core Concepts & Visual Blueprint

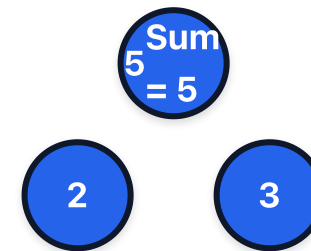
Theoretical models and memory architectures

---

## Key Principles

- **Sum Tree:** Node val is updated to sum of left & right children recursively.
- Return sum of tree node: `old_val + new_val`.

## Memory & Execution Blueprint



# C++ Implementation Reference

Standard code layout and syntax

---

```
int toSumTree(TreeNode* root) {  
    if (!root) return 0;  
    int old = root->val;  
    root->val = toSumTree(root->left) + toSumTree(root->right);  
    return old + root->val;  
}
```

# Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

## Performance Table

| Aspect / Case    | Complexity |
|------------------|------------|
| Time Complexity  | $O(N)$     |
| Space Complexity | $O(H)$     |

## Key Practices & Gotchas

- **Important:** Leaf nodes are updated to 0 because their child sums are zero.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

# Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

## Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

| Call Stack Level | Active Node Visited | Expected Left Traversal   | Expected Right Traversal   |
|------------------|---------------------|---------------------------|----------------------------|
| 1                | Root (1)            | Recurse on Left child (2) | Recurse on Right child (3) |
| 2                | Leaf (2)            | Base case (nullptr)       | Base case (nullptr)        |

# Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

---

## Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.